

Category: Developer Productivity / API Cost Analytics**Target users:** API-key agent users.

1. App idea and user problem

TokenTrail is an Android API-cost companion for students and independent developers who use API-key coding agents: **Codex with OpenAI**, **ZCode with general Z.ai API access** and a DeepSeek-side tool to be selected. The app has three parts: a **monthly tower-defence game** whose building resources come from the tokens the user has already spent, an **analytics** view that prices and compares the logged API usage behind those tokens, and a **community forum** carrying official model and pricing news. An **advice agent** sits inside the game: it reads the user's usage records and recent forum posts, then answers questions with evidence. TokenTrail **does not run, route or control** the coding agents.

Scope is **API usage priced at public API rates**; ChatGPT and GLM Coding Plan subscriptions are excluded. A response reports tokens for one call, not the agent's history, and an API key cannot recover another agent's past per-call usage, so a call is priced only when a saved rate snapshot covers its date. Every monetary value is an *estimate*, never an invoiced charge.

2. Related products and opportunity

The review compares first-party API billing views with existing cross-model monitoring tools on data scope, cost provenance and decision support.

Product	Relevant capability	Gap addressed by TokenTrail
Codex	Can run with an OpenAI API key, billed through the Platform account.	TokenTrail estimates only logged Codex API calls using dated OpenAI prices.
ZCode	Can use the general Z.ai API endpoint with an API key, separate from Coding Plan.	TokenTrail prices logged general-API calls, excluding plan usage.
dsh-context	DeepSeek Harness plugin showing local sessions, token use, cache, activity and estimated costs.	TokenTrail spans API providers; session detail appears only when usable logs are imported.
LiteLLM	Open-source gateway with routing, spend tracking and budgets for proxied calls.	TokenTrail does not proxy traffic; it tracks agent-associated API costs from permitted records.
dsh-pet	Interactive floating pet driven by DSH session events, with touch and feeding actions.	TokenTrail adds cross-provider, evidence-linked advice, and turns consumption into a visible monthly progression.

3. Proposed contribution

Unified view. TokenTrail merges authorised API-call logs from Codex, ZCode and a DeepSeek-side tool, removes duplicate records and shows source and coverage; session detail appears only where logs support it. **Reproducible costs.** Java applies dated official rates by provider, model, token class and service tier, and keeps the price version behind each estimate. **Reflected progress.** Tokens already spent on real work convert into building resources in a monthly tower-defence game, so a day of agent work leaves a visible result without asking the user to spend differently. **Decision support.** Activity, budgets and task ratings help users understand spending and choose an agent; an in-game advice agent is the conversational interface, its read-only tools supply calculated evidence, and the model states sample size and missing data. It neither controls coding agents nor interrupts the workflow. Values remain *Estimated*, *Partial estimate* or *Unavailable*.

Scope rule: The three core parts are the monthly tower-defence game, the API-cost analytics view and the community forum. The advice agent can recommend but cannot execute coding agents or enforce a hard cap. Subscription quotas, live cross-agent status and system-wide overlays are outside the first release.

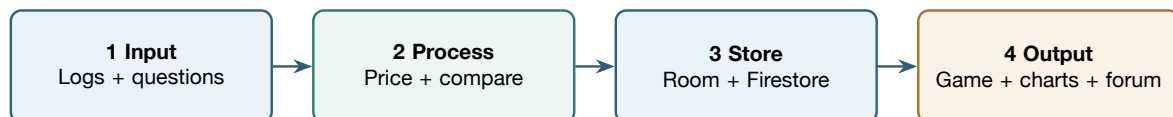
4. Planned functionality and the four data aspects

Aspect	Planned implementation	User value / evidence
Data input	Coding-agent profile, key label, budget and task rating; authorised usage logs; user questions to the in-game agent; forum posts.	Codex, ZCode and a DeepSeek-side tool; explicit source and coverage labels.
Data processing	Java prices dated usage, compares tasks and converts spent tokens into game resources; the agent requests read-only summaries and drafts advice.	Advice cites data and exposes missing fields; no invented live activity.
Data storage	Firebase Authentication; UID-restricted Firestore; Room for usage, price versions, game state and advice history.	Historic estimates retain the price version used at calculation time.
Data output	A monthly tower-defence game, Java/XML cards, heatmap, sessions, budget alerts and the community forum.	A mobile view of spending and work patterns, with unavailable states for missing logs.

5. Core features and implementation approach

- **Account and agent profiles.** Registration, login, logout and password reset. Users add Codex, ZCode and a DeepSeek-side tool as tracked profiles with a non-secret key label. No agent password is requested and no task is dispatched.
- **API usage and cost.** Import user-authorised per-call usage logs; no agent must route traffic through TokenTrail. Record call time, model, agent, provider and input, cached-input and output tokens where available, de-duplicate calls and apply the matching saved price version. CNY display uses a dated exchange-rate snapshot.
- **Session insights and efficiency.** When logs permit, group calls by session and show title, time, turns, tokens, cache hit, tool calls and duration; session cost is the sum of eligible call estimates. With only aggregate tokens, show provider-level estimates without inventing sessions.
- **Budget and action.** Compare estimated spend with a user budget, showing coverage and uncertainty; forecasts use comparable history only. Alerts are advisory, not a cap on external agents, and no payment is handled.
- **Monthly tower-defence game.** The home screen is a tower-defence game played in monthly seasons that match the billing cycle AI subscriptions follow. Tokens already spent convert into building resources at a fixed rate per million tokens; resources buy and upgrade defences, walls and a data-centre base that expands in one direction. A wave starts only when the user begins one and runs in real time, so no offline simulation or scheduled settlement is needed. A season resets each month.
- **In-game advice agent.** An agent inside the game opens an expandable question panel and reads two sources: the user's logged usage and recent forum posts. A Java service calls one selected external GPT, GLM or DeepSeek model, which may request *getUsageSummary*, *getBudgetStatus*, *compareAgents* and *getForumHighlights* through function calling; Java executes and validates these read-only tools. Replies state evidence and missing data, no logs means no cost-based answer, and the agent's own tokens are tracked separately from coding-agent usage.
- **API price updates.** Store official-rate snapshots with source URL, currency, model, token class, service tier and observation time. New rates apply prospectively and old estimates keep their version; where no contemporaneous rate exists, a current-price comparison is labelled uncertain rather than backdated.

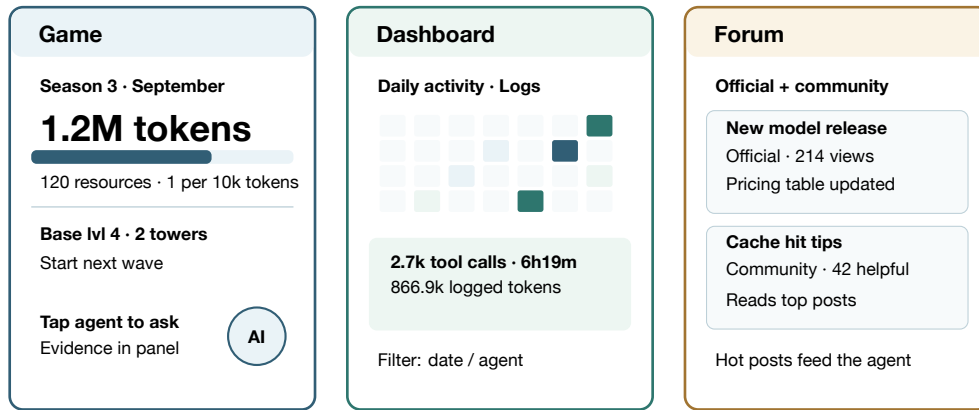
6. Data flow and limitations



Model responses report usage only for calls observed by the caller, so imported Agent logs are the primary input. Java matches token classes to dated prices, computes budgets and converts spent tokens into game resources; the external model receives selected summaries rather than API secrets or prompt text. A server-side function-call loop lets the agent query read-only tools, then return a short recommendation with source and coverage. The agent cannot observe a live coding session without a separate event feed.

7. Proposed Android interface

The app opens directly into the **game**; a bottom bar reaches the **forum** and the **dashboard**, which holds Activity, Sessions, Compare and Budget. The game leads because it is the part a user returns to daily, keeping the advice agent one tap away. Inspired by dsh-context and dsh-pet, these sketches show summary metrics, a heatmap and forum cards. Log-dependent sections show an empty state when records are unavailable.



Interaction and accessibility. Each screen shows loading, empty and error states. Figures carry Agent, provider, estimate status, usage coverage, price version and refresh time; the heatmap has a numeric summary. A context ring is not API spend. The team checks touch targets, text scaling and screen rotation.

8. Technology and responsible reuse

Layer	Planned tools and rationale
Android interface	Java/XML, Material Components and Navigation; ViewModel/LiveData for state; RecyclerView and MPAndroidChart for trends; a tower-defence game surface and the in-game agent panel.
Network and work	Retrofit/OkHttp for log import and a Java advice service. The service runs one model API and a bounded function-call loop over read-only usage, budget and comparison tools. WorkManager schedules price checks.
Persistence and security	Room plus UID-restricted Firestore; the Java service verifies Firebase ID tokens and holds its model key server-side. Imported metadata is protected; coding-agent passwords and keys are not required.
Open-source reuse	Room with a View guides storage; MPAndroidChart may supply charts; an open-source tower-defence sample may supply the wave and placement loop; dsh-pet inspires the agent's entry point. Original or properly licensed art will be used, and reused files and licences recorded.

9. Work plan through the final submission

The three-person team will follow the approximate course checkpoints: Alpha (weeks 8–9), Beta (weeks 11–12) and Final (weeks 15–16). The team will commit to GitHub throughout the project and adjust scope based on tested progress.

Course weeks	Work	Checkpoint / acceptance
1–3	Confirm API-key agent use cases and UI; set up Firebase Auth and Firestore rules; document usage-log access and pricing dimensions.	Neither test account sees the other's records; token-source limits documented.
4–7	Implement agent profiles, usage-log import, Room/Firestore records, versioned pricing, CNY display and the dashboard.	Each provider's sample calls are priced at the correct dated rate and de-duplicated.
8–9: Alpha	Deliver a working login → add agent → import log → estimated cost → analytics path; separate real logs from sample data.	Source, description, GitHub link and MP4 per submission guidance.
10–12: Beta	Add budgets, sessions and task ratings; build the tower-defence loop (season, token-to-resource conversion, user-started waves) and the agent panel; connect one model to three read-only tools.	The agent answers a usage question with evidence and admits missing data; a wave starts and resolves without blocking the UI.
13–15: Final	Evaluate advice accuracy and API cost, balance the season, improve accessibility, verify account isolation and document estimation limits.	End-to-end demo, report and video; no agent answer invents usage or controls coding agents.

10. Team work and evidence of completion

Split by the three core parts. **Zhang Li (24107757):** UI/UX, Java/XML screens, forum screens, charts, accessibility. **Wang Tingdong (24107759):** authentication, usage-log import, Room/Firestore, versioned pricing, budget computa-

tion. **Liu Zongrun (24107745)**: the tower-defence game, the server-side agent service, its read-only tools and advice evaluation. Forum data each part consumes is agreed jointly; all three review integrations and commit to GitHub.

The demo shows account isolation; tracked Codex, ZCode and DeepSeek-side profiles; three provider estimates from dated token logs; a season in which those tokens became build resources; a heatmap and budget warning; then an agent question such as “Why did cost rise?” answered with a queried summary, evidence and its own estimated API cost.

Validation tests token accounting, price-version boundaries, missing usage, de-duplication and read-only tool limits, with the Firebase Emulator verifying access. Agent replies are checked against tool data, and the game is checked for a resource balance unreachable without logged usage. Published rates can differ from paid amounts; all figures remain estimates.

11. Functional requirements and improvements

Each capability is mapped to what comparable tools already offer, what TokenTrail changes, and what is new. The advice agent runs inside the game.

Ser	Existing features	Proposed improvements	New features	Remarks
1	Cross-model cost dashboards	Proxy- or provider-scoped, no dated rate snapshot	Three providers priced from dated official rates, de-duplicated, with source and coverage	Always an estimate
2	Chat-history viewers	Show messages, not priced token classes	Per-call and per-session token-class breakdown, cache read and write split	Needs an authorised log
3	Desktop context dashboards	Desktop, single user, no accounts	Android client with Firebase Auth and UID-scoped rules	Emulator-tested
4	Interactive pet (dsh-pet)	Scripted reactions to session events	Monthly tower-defence season, one-direction base growth, a wave the user starts	Resources from tokens already spent
5	General chat assistants	No access to the user's own usage data	In-game agent with function calling over read-only tools; states sample size and missing data	Reads usage and hot forum posts
6	Provider billing consoles	Budgets are provider-scoped	One cross-provider monthly budget with alerts and per-session breakdown	Against a user-set CNY budget
7	Community forums	Separate from usage data	Official model and pricing posts plus user tips, ranked for the agent	Read-only to the agent
8	Task comparison	Absent from existing monitors	Rate comparable tasks per agent to guide the next choice	Needs enough logged sessions

12. Your key contributions and improvements

I build the parts TokenTrail does not take from a reference project, and integrate them with my teammates’ dashboard and forum work:

- **Monthly season model.** Converting tokens already spent into build resources, with a season boundary that follows the billing cycle so progress and spend reset together.
- **Tower-defence loop.** Real-time rendering, one-direction base growth, and waves the user starts rather than a timer firing while the app is closed.
- **The advice agent.** The tool schema, the prompt, and a bounded server-side function-call loop over four read-only tools, so every answer cites queried data instead of recalled numbers.
- **Server-side advice service.** A Java service that verifies Firebase ID tokens and keeps the model key off the device, so the APK ships no usable API secret.
- **Cross-part interfaces.** The contract letting the game read dashboard figures and forum highlights without coupling the three parts’ storage.

13. Sources and project references

API scope	OpenAI pricing; DeepSeek pricing; Z.ai pricing; Codex billing modes; ZCode endpoints.
Agent	Function calling; Java API client; Firebase ID-token verification.
Related	dsh-context dashboard; dsh-pet; LiteLLM; Room with a View; MPAndroidChart.
Security	Firestore Security Rules conditions.